

Octagonal Gaming Table Turn Counter — Design Document

Project	LED rim turn counter for an octagonal gaming table
Geometry	8 sides × 20" each = 160" (~4 m) perimeter
Player count	Variable, 2–8 (configurable in firmware setup mode)
Input method	Piezo sensor per side (slap-to-advance)
Brain	ESP32-S3 (with optional OTA firmware updates over Wi-Fi)
Target	Reliable, satisfying turn-passing mechanic

Companion files: - `dry_run.md` — Phase –1 bench & skills shakedown on penny parts; do this first - `turn_counter_wiring.svg` — full schematic, print and keep at the bench - `doc-src/breadboard_layout.svg` — hole-by-hole breadboard placement for the **Phase 2 bench prototype** (it includes the level shifter — the Phase 0 Tap Light deliberately skips it) - `turn_counter.ino` — main project firmware - `tap_light.ino` — Phase 0 starter project firmware

Other diagrams (top view, edge cross-section, slab/frame architecture) are embedded inline in the relevant sections of this document.

Summary

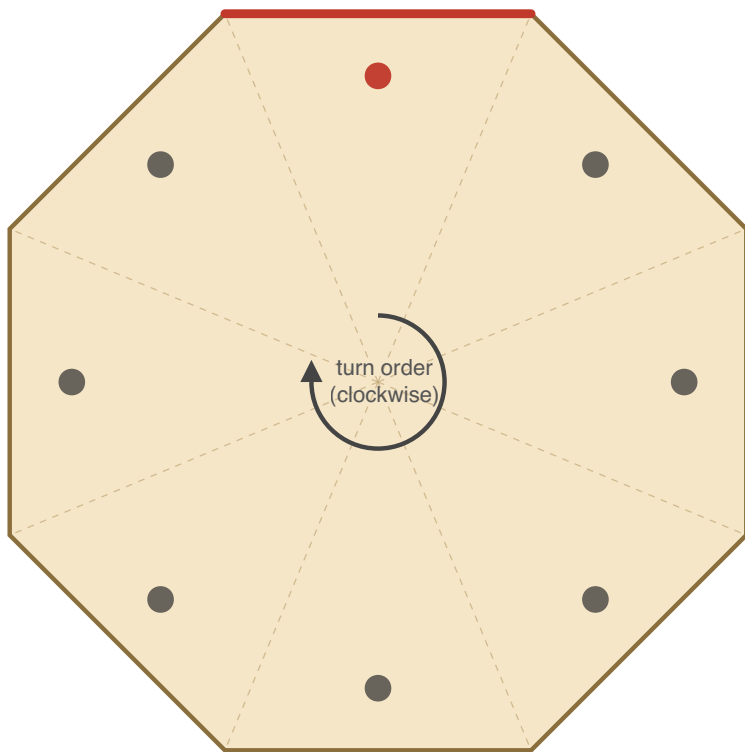
A perimeter LED rim around an octagonal gaming table shows whose turn it is — one section glows in the active player's color while the rest stays dark. The active player taps their section to pass turn; the lit zone advances clockwise to the next player. Player count is configurable from 2 to 8, and a two-handed slap on opposite sides toggles the whole thing on or off.

The build is a thin octagonal slab that sits on top of an existing bumper pool table. The slab is removable — a single DC connector pops apart so it can lift away, leaving the PSU and AC mains permanently mounted to the pool table frame underneath.

This document walks through the build end-to-end: tools and a Phase 0 starter project for first-time soldering, bill of materials, electrical and mechanical design, step-by-step assembly, firmware reference, and troubleshooting.

Octagonal Table Layout (Top View)

LED channel on outer edge, piezos on underside



● active piezo (current player)

● inactive piezo (taps ignored)

— lit LED zone (active player)

- - - wedge boundary

Opposite-side pairs trigger the on/off gesture when slapped simultaneously

Figure 0.1 — top-down view of the finished table. The lit edge indicates the current player's section; tapping that section advances the turn clockwise to the next player.

0. Before You Start: Tools, Skills, and a Practice Project

This is your first electronics project. Don't skip this section. Going straight to the main build means doing 40+ solder joints on parts you can't easily replace, while learning to solder, on a project that won't work if any one of them is bad. That's a bad time.

Phase 0 has three goals:

1. Get you a real bench setup with the tools you'll need.
2. Teach you the techniques (soldering, multimeter use, ESP32 flashing) on a small, forgiving project.
3. Leave you with a finished, working **tap-activated desk light** that uses the same parts and patterns as the main build, just at 1/8 scale.

Plan on **4–6 hours** for Phase 0 spread over a couple of evenings. First-time soldering is slow, and that's fine.

Even before Phase 0, there's a ~2–3 hour **Phase –1 dry run** in `dry_run.md`. It shakes down your bench, multimeter, soldering, and (optionally) the ESP32 toolchain on penny parts from your ELEGOO kit — no strip, no PSU, no expensive components on the bench. If this is genuinely your first time holding an iron, do it first: it means any problem on the Tap Light is a wiring issue, not a "does my equipment even work" mystery.

0.1 Tools You'll Need

Essential (~\$80–120 total):

Tool	Recommendation	Notes
Soldering iron with adjustable temp	Pinecil V2 (~\$30) or Hakko FX-888D (~\$110)	Skip the \$15 fixed-temp pencil irons. Variable temp is the difference between fun and frustration
Solder	60/40 or 63/37 leaded rosin core, 0.6 mm or 0.8 mm	Leaded is much easier to learn on (63/37 is eutectic — snaps solid instantly, even fewer cold joints). Ventilate and wash hands. Lead-free is harder and not necessary at this scale
Side cutters / flush cutters	Hakko CHP-170 (~\$8)	For trimming component leads
Wire strippers	Any self-adjusting strippers (~\$15)	Not the manual ones with a sliding stop
Multimeter	Any cheap one (~\$20)	You only need continuity and DC voltage. AstroAI on Amazon is fine

Strongly recommended (~\$30 total):

Tool	Recommendation	Notes
Helping hands or PCB vise	Anything with alligator clips and a vise	Genuinely transformative. Soldering with no third hand is miserable
Flux pen or paste	Kester 951 or MG Chemicals	Makes solder flow properly. You need this
Heat-shrink assortment	\$10 mixed pack	Insulating splices and joints
Lighter or hot air gun	Any Bic	For heat-shrink. Hair dryer doesn't get hot enough
Solder wick / desolder braid	\$5	For fixing mistakes (you will make some)
Bench mat	Silicone, ~\$15	Heat-resistant, prevents you from burning your desk

Nice to have, but skip for now: fume extractor, hot air rework station, magnifier, "third hand" with magnifier.

0.2 Soldering Primer

The skills are mostly muscle memory. Watch a video before you start — these two are good:

- "How to Solder" by EEVblog (12 min — fundamentals)
- "Common Soldering Mistakes" by Branchus Creations (8 min — what to avoid)

The condensed version that text can convey:

1. **Set the iron to ~330°C / 625°F** for leaded solder. Higher for lead-free, but you're using leaded.
2. **Tin the tip** when hot — touch a bit of solder to the tip, wipe on a damp sponge or brass coil. The tip should look shiny silver. A black, dull tip transfers heat poorly.
3. **Heat the joint, not the solder.** Touch the iron tip to both pieces being joined. Wait ~2 seconds. Then feed solder into the joint *from the opposite side of the iron*. The solder should flow toward the heat and wet both surfaces.
4. **Pull the solder away first, then the iron.** Don't move the joint while it cools (~1–2 seconds).
5. **Good joint:** shiny, smooth, slightly concave, wets both surfaces. **Bad joint:** dull, blobby, ball-shaped, only stuck to one side ("cold joint").
6. **If a joint is bad**, add a touch of flux, reheat, and either reflow or remove with wick and redo.

Warmup exercise before touching real parts: strip 4–5 pieces of scrap wire (any gauge), twist pairs together, and solder them. You'll do ~10 joints. By the last one, they should look noticeably better than the first. Don't proceed to real components until you can produce consistent shiny joints.

0.3 Multimeter Primer

You need exactly two functions for this project:

Continuity (often a  or diode symbol). Probes touched together = beep. Use it to:

- Verify a wire goes from where you think it does to where you think it does
- Check that solder joints actually connected
- Confirm a fuse is intact

DC voltage (V with a straight line, often "V=" or "VDC"). Set the range to 20V or use auto-range. Use it to:

- Verify a 5V power supply is putting out 5V (and not 12V because you grabbed the wrong PSU)
- Check polarity (red probe to +, black to GND, expect a positive number)

Critical habit: every time you connect power to something new, multimeter the polarity at the destination side first. Reversed 5V on a WS2812B strip kills LEDs instantly and silently.

0.4 Arduino IDE Setup

Before any soldering, get the dev environment working:

1. **Install Arduino IDE 2.x** from arduino.cc.
2. **Add ESP32 board support:** File → Preferences → Additional Board Manager URLs → add `https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json`. Then Tools → Board → Boards Manager → search "esp32" → install "esp32 by Espressif Systems". (Same package supports the ESP32-S3.)
3. **Install libraries:** Tools → Manage Libraries → install **FastLED** (by Daniel Garcia). For the main project later, you'll also install **ArduinoOTA** (already bundled with the ESP32 core, no separate install).
4. **Plug in your ESP32-S3 dev board** via USB-C. Most DevKitC-1-style boards have **two** USB ports and either one flashes the chip: the port labeled **USB** is the S3's native USB (enumerates as a USB CDC device, `/dev/cu.usbmodem*` on macOS — no driver to install); the port labeled **UART/COM** goes through a CP2102 bridge (`/dev/cu.usbserial-*` — modern macOS and Windows ship the driver). Serial output follows the **USB CDC On Boot** setting: Enabled → native port, Disabled → UART port.
5. **Test upload:** File → Examples → 01.Basics → Blink. Tools → Board → ESP32 Arduino → "**ESP32S3 Dev Module**". Tools → **USB CDC On Boot: Enabled** (so Serial reaches the IDE over the native USB port). Tools → Port → select the new port that appeared when you plugged in the board. Click upload. If your board has an onboard RGB LED, the Blink example won't drive it — try `Examples → ESP32 → ChipID` to confirm the chip responds.

If upload fails: hold the BOOT button on the board, tap RESET, release BOOT, then upload. Drop the upload speed to 115200 in Tools → Upload Speed if it still fails.

Before starting the Tap Light: if you haven't done the **Phase –1 dry run** (`dry_run.md`), do it now. It confirms your iron, meter, soldering, and the ESP32 toolchain all work — on throwaway parts — so the Tap Light is the first time those skills meet real components, not the first time you find out a tool is broken.

0.5 The Starter Project: Tap Light

A small WS2812B accent light with a single piezo sensor. Tap it to cycle through 6 modes (warm orange, blue, green, amber, rainbow, off). Power it from the ESP32's USB port — no separate PSU, no mains wiring, no level shifter to start.

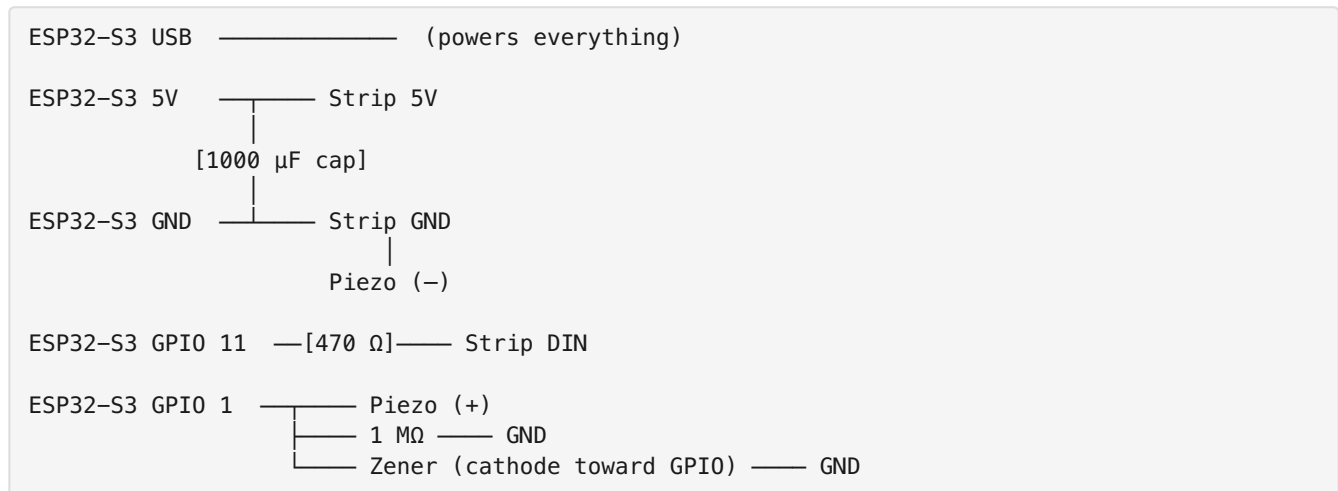
This is essentially **one zone of the main project**. Every skill you exercise here transfers directly.

Parts (all from the main BOM)

Qty	Part
1	ESP32-S3 dev board
30 LEDs	WS2812B strip (cut from the 5 m roll — you have plenty)
1	Piezo disc
1	1 M Ω resistor
1	3.3 V Zener diode
1	470 Ω resistor
1	1000 μ F capacitor (optional at this scale, but install it for practice)
~	Hookup wire, heat-shrink
1	Breadboard (~\$5, get one anyway, useful for life)
1	USB cable for the ESP32

Skip the level shifter for the starter. WS2812B at short lengths often works fine on 3.3 V data directly — and if it doesn't, that's a teaching moment about why we add the level shifter for the main build.

Wiring



This is identical to the main wiring diagram, just with one piezo channel and no level shifter.

Build Steps

Session 1: Solder practice + LED strip prep (1–2 hours)

- ☐ Set up your bench: iron heating, solder, helping hands, multimeter, scrap wire
- ☐ Warmup: 5 wire-to-wire solder joints on scrap. Inspect each. Don't proceed until they look good
- ☐ Cut a 30-LED segment from the strip (between LED pads, follow the cut lines printed on the strip)
- ☐ Tin the three pads at one end of the segment (5V, DIN, GND). Add solder until each pad has a small dome

- ☐ Strip ~4 mm of insulation off three short hookup wires (red for 5V, black for GND, any other color for data)
- ☐ Tin the wire ends
- ☐ Solder each wire to its pad. Touch iron to pad, melt the dome, slide wire in, hold steady, remove iron. ~2 seconds per joint
- ☐ Continuity test: probe each wire end against the matching pad on the strip. Should beep
- ☐ Heat-shrink each joint individually for strain relief

Session 2: Build the input network and breadboard (30–60 min)

- ☐ Solder leads to the piezo: red to (+), black to (–). Be **fast** — under 2 seconds per pad. Piezo discs lose sensitivity when overheated
- ☐ On a breadboard, lay out the circuit per the wiring above
- ☐ Place the ESP32 across the breadboard's center channel
- ☐ Use jumper wires for the ESP32-to-component connections
- ☐ The 1 MΩ resistor sits between GPIO 1 and GND
- ☐ The Zener sits between GPIO 1 and GND, with the **black band (cathode) toward GPIO 1**, body toward GND. Polarity matters
- ☐ Don't connect power yet

Session 3: Test and tune (1 hour)

- ☐ Visual check: walk the wiring against the schematic one more time
- ☐ Multimeter polarity check: with USB unplugged, probe ESP32 5V to GND with continuity — should NOT beep (no shorts)
- ☐ Plug in USB. The onboard power LED should light (most ESP32-S3-DevKitC boards have one near the USB port). If you smell anything or the board gets warm, unplug immediately and check for shorts
- ☐ Open Arduino IDE, load `tap_light.ino`, upload
- ☐ Strip should light up in the first mode (warm orange). The onboard RGB pixel mirrors the current mode too, so tap → color cycling is testable even before the strip is wired
- ☐ Open Serial Monitor at 115200 baud — at boot it prints the piezo baseline (the sketch averages the resting level for ~0.5 s, then keeps tracking it slowly)
- ☐ Tap the piezo. Mode should advance, and a line should print to serial showing the reading, the baseline, and the delta between them
- ☐ If no response: tap harder, or lower `TAP_DELTA` in the code and re-upload. Detection is adaptive — a tap fires when a reading jumps `TAP_DELTA` (default 1500) above the tracked baseline, so noise and drift are absorbed automatically; `TAP_DELTA` only sets how hard a hit must be
- ☐ Cycle through all 6 modes by tapping. Confirm the rainbow mode looks right (each LED a different color)
- ☐ Unplug USB, plug back in. The strip should resume on whatever mode you left it on (state persists in NVS)

Session 4 (optional): Move from breadboard to protoboard, add an enclosure

If you want a finished thing rather than a breadboard prototype:

- ☐ Lay out the circuit on a small protoboard (5×7 cm)

- ☐ Use female headers for the ESP32 so you don't solder it directly
- ☐ Solder the resistor and Zener through-hole; trim leads with side cutters
- ☐ Run wires to the strip and piezo via JST connectors or solder direct
- ☐ Mount in a small project box, or hot-glue everything to a piece of scrap wood

0.6 Skills Check

If you can honestly check all of these, you're ready for the main build.

- ☐ My solder joints look shiny and properly wetted (not blobby or dull)
- ☐ I can solder a wire to an LED strip pad without lifting the pad
- ☐ I soldered a piezo without killing it
- ☐ I know how to test continuity with my multimeter and have done so on real circuits
- ☐ I successfully uploaded firmware to the ESP32 via USB
- ☐ I read serial output and used it to tune a parameter (the piezo tap delta)
- ☐ I understand the per-piezo input network well enough that I could draw it from memory
- ☐ My Tap Light works end-to-end and persists state across power cycles

If any of these are no, repeat the relevant section before moving on. The main build has 8x the joints, mains wiring, and corner soldering on a strip mounted in aluminum channel — none of which is forgiving.

1. The Broad Plan

The main build splits into seven phases on top of Phase 0. Do them in order.

Phase	What happens	Why it's here
–1. Dry run	Bench + skills shakedown on penny parts (see <code>dry_run.md</code>)	Prove your tools and hands work before any expensive part is on the bench
0. Starter project	Build the Tap Light (see §0)	Skills, bench setup, confidence
1. Plan & gather	Parts list ordered, work area set up	Nothing worse than getting halfway and realizing you're missing a Zener
2. Bench prototype	ESP32 + 1 LED segment + 1 piezo on a breadboard, firmware running	Proves the full firmware works before anything is permanent
3. LED rim	Cut, route, and mount the strip into aluminum channel around the octagon	The most visible part of the build. Take your time on the corners
4. Piezo mounting	Glue 8 piezos to the underside of the slab, run leads	Cross-talk between sides is inherent to a single slab; firmware filters it, but careful placement helps
5. Control box	ESP32 + level shifter + resistors + connectors on protoboard, in an enclosure	Everything terminates here. JST connectors so the table is serviceable
6. Final assembly	Mount PSU on the bumper pool frame, wire the Powerpole disconnect, dress wiring on the slab, mate and test	The slab/frame split (§4.6) is most of the work here
7. Tune & test	Dial in the piezo tap delta, calibrate brightness, play a real game	Real-world testing always reveals something the bench didn't

Estimated time: ~2–3 hours for the Phase –1 dry run, 4–6 hours for Phase 0, plus 10–14 hours for Phases 1–7 spread over a weekend. Most of the main build is mechanical (channel cutting, mounting), not electronics.

2. Bill of Materials

Quantities below cover the **main project**. The starter project (Phase 0) uses a subset — see §0.5. Order everything together; the per-unit prices on extras are negligible and you'll appreciate having spares.

Qty	Part	Specifics	Notes
1	ESP32-S3 dev board	ESP32-S3-DevKitC-1, N8R8 variant (8MB flash + 8MB octal PSRAM), USB-C native	Buy 2 — one for the starter, one for the main. ~\$15 each. N8 and N8R2 are end-of-life at DigiKey; N8R8 is the current stocked variant. Octal PSRAM uses GPIOs 35/36/37 internally — fine for this build (the pinout in §3.1 doesn't touch them), just don't reassign anything to those pins. Get the PCB-antenna part (no U suffix); the -1U-N8R8 variant has a U.FL connector and no antenna
5 m	WS2812B LED strip	60 LEDs/m, IP30, 5V	4 m used; the extra is for the starter and mistakes
4 m	Aluminum LED channel	With frosted/diffuser cover	Massively improves the look
10	Piezo disc	27 mm brass, 2 leads	Bag of 10 covers main + starter + spares
12	Resistor	1 M Ω , 1/4 W	Pulldown across each piezo. Buy a 100-pack, they're nothing
12	Zener diode	3.3 V, 1 W (1N4728A)	ADC overvoltage clamp
5	Resistor	470 Ω , 1/4 W	Series resistor on data line
3	Capacitor	1000 μ F, 10 V or higher, electrolytic	Across 5V/GND at strip start
2	Level shifter	74AHCT125 (DIP-14)	3.3 V $\rightarrow$ 5 V data line
1	Power supply	Mean Well LRS-100-5 (5 V, 18 A)	Don't cheap out here. LRS-50-5 (5 V, 10 A) is the original spec and also fine if you can find it; the -100 was substituted when the -50 went out of stock at DigiKey
1	Power switch	Panel-mount rocker rated for your mains ($\geq$ 125 V AC covers US 120 V; 250 V AC if outside the US), 6 A+	Or use a switched IEC inlet (safer)
~	Hookup wire	22 AWG stranded for signal, 18 AWG for power	A few colors helps
12	JST-XH 2-pin connector pairs	For piezo leads	Makes service easy
2	JST-XH 3-pin connector pair	For LED strip data + ground	Detachable rim

Qty	Part	Specifics	Notes
1 pk	WAGO 221-415 lever connectors	5-conductor, 25-pack	Branch nodes for the slab DC rail (control-box feed + 3 strip injection points). Replaces the screw-terminal blocks of earlier drafts
1	Project box	~110×60×30 mm	See §4 for mounting options
2	Protoboard	5×7 cm or similar	One for starter, one for main
1	Inline fuse	5 A automotive blade fuse + holder	Between PSU 5V output and Powerpole pigtail
1	Breadboard	Standard half-size	For prototyping. Useful forever
	Removable-top installation (\$4.6)		
3	Anderson Powerpole 30 A connector kit	Pair of red + black housings + contacts	One pair for the slab, one for the frame side it mates with, one for a bench-test pigtail (see §4.6). Contacts get soldered + heat-shrunk, not crimped (procedure in <code>shopping_list.md</code>) — no crimp tool needed
6 ft	Silicone-insulated wire	14 AWG, red and black (3 ft each)	DC run from PSU to slab
1	Soft rubber grommet	½" or ⅝" panel-hole size; inside diameter sized to your DC cable (the 14 AWG silicone twisted pair is $\sim \frac{5}{16}$ " / 8 mm OD)	Cable entry through slab. Hardware-store generic — <i>avoid</i> sheet-metal-panel bushings like Heyco 2092, which are designed for 3 mm panels, not 1" wood
2	P-clips or adhesive cable mounts	Various sizes	Cable strain relief inside slab
1	Pine block or scrap	~4×6×1"	PSU mounting platform on frame
~	#8 wood screws	¾" and 1.5" lengths	Mount block to frame and PSU to block

Total cost: ~\$271–281 for parts, plus ~\$213 for tools if you own none — `shopping_list.md` has the vendor-priced, per-part accounting. Tools are a one-time investment that'll serve every future project.

Where to source: Strip and channel from BTF-Lighting or Adafruit. Piezos, resistors, Zeners, level shifter from any electronics supplier (DigiKey/Mouser if you want quality, Amazon if you want speed). PSU from Mean Well's authorized resellers — there are a lot of fakes on Amazon.

AC mains wiring note: the BOM uses a **discrete-component approach** — a separate IEC C14 inlet and panel-mount SPST switch wired in series on the Line conductor between the wall plug and the PSU. The wall plug feeds the IEC inlet's Line and Neutral pins; Line goes through the SPST switch to the PSU's L

input; Neutral runs from the IEC inlet direct to the PSU's N input (no switch); Earth runs from the IEC inlet's Earth pin direct to the PSU's chassis Earth lug. **The BOM's inline 5 A blade fuse is a DC-side part** — it goes between the PSU's +5 V output and the Powerpole pigtail (§3.3), never on the AC line: automotive blade fuses are rated 32 V DC and cannot safely interrupt 120 V AC. The AC side needs no added fuse — the LRS-100-5 carries its own internal input fuse. With the switch off, Line is interrupted so no current can flow — same approach as every wall switch in your US household. **Important safety habit:** unplug from the wall before opening the enclosure. Never rely on the switch alone as service isolation — only the wall plug guarantees both Line and Neutral are disconnected. The alternatives — an integrated switched/fused IEC inlet (Schurter DG12 series, ~\$46) or a DPST switch that interrupts both Line and Neutral — are nice-to-haves, not requirements. The SPST + "unplug before service" discipline is the residential-wiring standard.

Octagonal Turn Counter — Wiring Diagram

8 piezos • ESP32-S3 • WS2812B (240 LEDs) • 5V/18A PSU

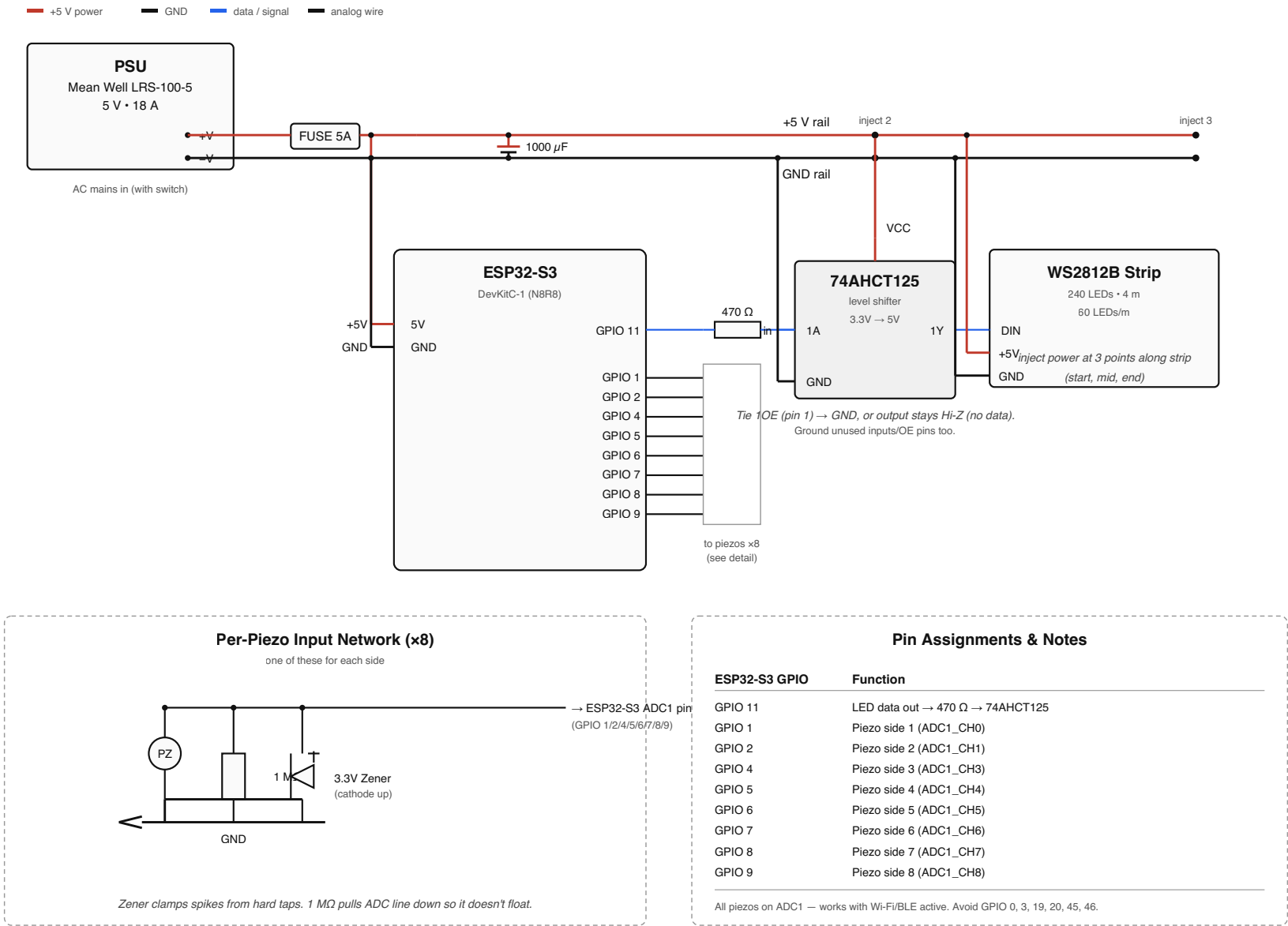


Figure 3.1 — full wiring diagram. PSU at top-left feeds +5V/GND rails across the top. ESP32 drives 8 piezo inputs and one LED data line through a 470 Ω resistor and 74AHCT125 level shifter. Three power injection points distribute current along the strip. Per-piezo input network detail at bottom-left.

3. Electrical Design

See `turn_counter_wiring.svg` for the full schematic. This section summarizes; the SVG is authoritative.

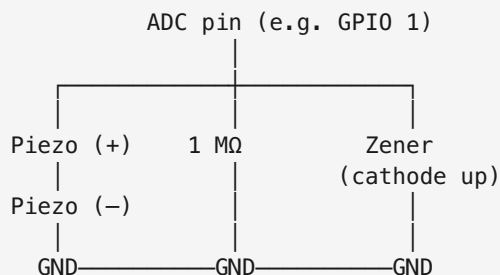
3.1 Pinout

ESP32-S3 GPIO	Function	Notes
11	LED data out ($\rightarrow 470\ \Omega \rightarrow$ level shifter $\rightarrow$ strip)	
1	Piezo side 1 ADC	ADC1_CH0
2	Piezo side 2 ADC	ADC1_CH1
4	Piezo side 3 ADC	ADC1_CH3
5	Piezo side 4 ADC	ADC1_CH4
6	Piezo side 5 ADC	ADC1_CH5
7	Piezo side 6 ADC	ADC1_CH6
8	Piezo side 7 ADC	ADC1_CH7
9	Piezo side 8 ADC	ADC1_CH8

All 8 piezos are on **ADC1**. This matters: the ESP32-S3 radio takes over **ADC2** when Wi-Fi or BLE is active, and any `analogRead` on an ADC2 pin silently returns 0. Sticking to ADC1 (GPIOs 1–10) means tap detection keeps working with OTA, a phone web UI, or any future BLE feature.

Avoid GPIO 0, 3, 45, 46 — strapping pins; pulling them at boot changes boot mode (GPIO 0/46), flash/VDD_SPI voltage (GPIO 45), or the JTAG signal source (GPIO 3). GPIO 19 and 20 are the native USB D-/D+ lines — leave them untouched. GPIO 35/36/37 are reserved on Octal-PSRAM modules (N8R8 / N16R8) — and since the BOM now specs the N8R8 (N8/N8R2 are EOL at DigiKey), those three pins are off-limits for this build. The current pinout above doesn't use them, so this is only a constraint if you add features later.

3.2 Per-piezo circuit (×8)



The 1 M Ω resistor pulls the ADC line down so it doesn't float. The Zener clamps spikes — a hard slap on a piezo can produce 20+ volts momentarily and the ESP32-S3 ADC tops out at 3.3 V.

Optional hardening: a 10 k Ω –47 k Ω resistor in series between the piezo/clamp node and the ADC pin. The piezo's negative half-swing forward-biases the Zener at about -0.6 V, slightly past the S3's -0.3 V absolute-maximum rating. The classic knock-sensor circuit survives this in practice (microjoule pulses from a very high-impedance source), but the series resistor limits the injected current to tens of microamps and makes it unambiguously safe — and costs nothing for tap detection, since the ADC input draws no current.

3.3 LED strip + power

Three injection points along the strip — start, middle, and end — each tapping the +5V and GND rails on the slab. This prevents voltage droop along the 4 m run. Note that those rails on the slab come from the PSU on the bumper pool frame via the Powerpole DC disconnect; see §4.6 for the full slab/frame architecture. The schematic shows the electrical connections; the disconnect itself is a physical break in the +5V/GND wiring between PSU and slab.

The 74AHCT125 level shifter takes the ESP32-S3's 3.3 V data signal and bumps it to 5 V. Tie its OE pin (and any unused channel OE pins) to GND. Power VCC from +5V rail.

A 1000 μ F cap across the 5V/GND rails right where the strip starts. Acts as a local energy reservoir for sudden current draw.

An inline 5 A fuse between PSU 5V output and the rest of the system. If something shorts, the fuse blows instead of the strip.

3.4 Power budget sanity check

- 240 LEDs $\times$ 60 mA worst case (full white) = 14.4 A
- One side (30 LEDs) lit at 50% brightness, single color = ~ 0.5 A
- Idle (lights off, ESP32-S3 only) = ~ 30 – 50 mA without Wi-Fi, ~ 80 – 120 mA with Wi-Fi up for OTA

An 18 A PSU (LRS-100-5) is generous — even a 10 A PSU (LRS-50-5) would be comfortable, since we'll never light more than ~ 30 LEDs at once during normal play. The extra headroom on the -100 means the PSU itself won't enter overcurrent protection even if the firmware ever drives full-brightness white across the full strip (~ 14.4 A theoretical max), so the **inline 5 A fuse is what protects downstream wiring on a short** — don't skip it.

4. Mechanical Design

4.1 LED channel layout

The aluminum LED channel mounts to the **outer edge of the slab**, running around the entire octagonal perimeter. The diffuser faces outward so the glow is visible from any seat at the table.

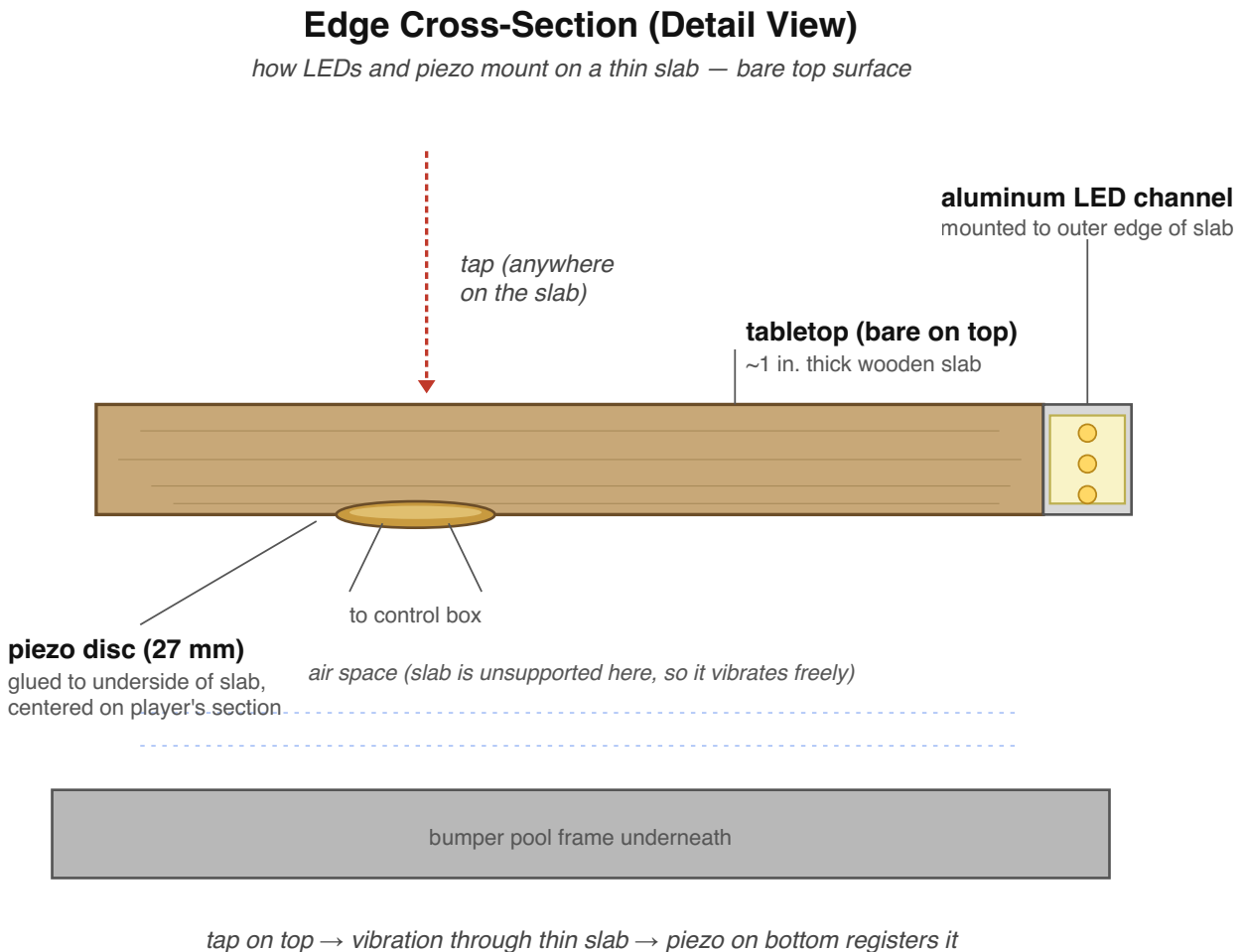


Figure 4.1 — edge cross-section. The aluminum LED channel mounts to the outer edge of the slab; the piezo glues to the underside, centered on each player's wedge of the octagon. Air space below means the slab vibrates freely, so a tap anywhere on the bare top surface couples efficiently into the piezo underneath.

The slab is thin (~1") and rests on the bumper pool frame with air space between, which means it vibrates freely. This is exactly what lets piezos on the underside register taps from anywhere on the slab.

Alternative: if you prefer a recessed look, route a 1/2"-wide × 1/4"-deep rabbet into the outer edge of the slab and recess the channel flush. More work, looks cleaner. Your call.

4.2 Corner handling

8 sides means 8 closed-loop interior corners. At each corner:

- **Cut the channel** at 22.5° miters with a hacksaw or chop saw. The angle for a regular octagon is 135° interior, so each piece gets a 22.5° cut on each end.
- **Cut the LED strip** between LED pads at the corner. Solder short jumper wires (3-conductor: 5V, GND, Data) to bridge across the corner gap. Heat-shrink the joints.
- **LED count per side**: at 60 LEDs/m, a 20" (508 mm) side fits ~30 LEDs. Cut cleanly between pads, you'll lose maybe 1–2 LEDs in the corner gaps.

4.3 Piezo placement

One piezo glued centered on the **underside of the slab**, one for each player's section (8 total). Brass face against the wood, leads soldered before mounting.

Layout: divide the underside of the octagonal slab into 8 wedge-shaped sections matching the 8 sides above. Place each piezo at roughly the centroid of its wedge — about 3–4" inward from the outer edge. Avoid placing them where the slab rests on the bumper pool frame; a piezo at a clamped point won't vibrate freely.

The thin slab + air-gap-below configuration is excellent for piezo coupling. Vibration from a tap anywhere on a player's section transmits efficiently to the piezo directly underneath. The bare top means nothing damps the slab from above either.

Mounting: hot glue is fine for prototyping. For permanent installation use cyanoacrylate (super glue) or a thin layer of epoxy. Press flat, hold 30 seconds. Don't crush the disc — even pressure across the whole face.

Wire routing: 22 AWG stranded leads from each piezo, soldered to the disc terminals (be quick — piezos are heat-sensitive), routed along the slab underside toward the control box. Secure with adhesive cable mounts every 6". From there to JST-XH connectors.

4.4 Cross-talk mitigation

Because the whole slab is one continuous piece of wood, every tap reaches all 8 piezos to some degree. The piezo directly under the tap reads strongest; piezos on adjacent sections read moderately; piezos on the opposite side read very weakly.

First-line mitigation: `readPiezos()` picks only the side with the *biggest jump above its own baseline* per scan as the accepted hit (the one exception is when the diametrically-opposite side also spikes in the same scan — that's the two-handed slap path). Adjacent cross-talk loses to the real hit automatically. Detection is adaptive: each side's resting level is averaged at boot and tracked slowly, and a hit fires at baseline + `TAP_DELTA` — so per-side differences in piezo sensitivity and mounting are absorbed rather than fought with one global threshold.

If you're getting false-side detection (a tap on side 1 occasionally registers as side 2):

- Raise `TAP_DELTA` so weak cross-talk doesn't reach the fire level to begin with

- If that's not enough, edit `readPiezos()` to require the winning side's jump to be at least 2× the second-strongest side's jump before accepting the hit (you'll need to track the second-best delta alongside the best inside the scan loop)

On the on/off gesture specifically: this configuration is *especially good* for opposite-side detection.

Cross-talk falls off with distance through the slab, and opposite sides are physically as far apart as possible. A single hard tap will never produce strong readings on opposite piezos — the slab simply doesn't transmit cross-talk that far at strong amplitudes. So "two strong readings exactly 4 sides apart" is unambiguously a deliberate two-handed slap.

4.5 Control box: enclosure options

Three choices, ranked from least to most effort:

Option A — Project box, surface-mounted under the slab (default). A small ABS or aluminum project box (~110×60×30 mm) screws to the underside of the slab. Cable glands or grommets for entries. Cheapest, fastest, easiest to service. Slight visual penalty if you ever flip the slab over.

Option B — Recessed pocket in the slab underside. Route a rectangular pocket into the underside of the slab, sized to fit the protoboard + ESP32 + connectors. Cover plate (wood or thin metal) screws over it. Flush, invisible from below, but you have to commit to a location and dimension before routing — and any future hardware change means another router pass. Pay attention to slab thickness: with a ~1" slab and a typical protoboard depth of ~10–15 mm, you have very little wood left between the pocket bottom and the top surface. Best if you're confident in the design and the protoboard is thin enough.

Option C — External enclosure on the bumper pool frame. If you don't mind a small box mounted to the frame underneath rather than on the slab, you can keep the slab clean. Trade-off: now you have signal cables (8 piezo lines + 1 strip data line) crossing the disconnect along with power, which means a much larger multi-pin connector instead of the simple Powerpole DC pair. Not recommended for this build, but worth knowing it's an option if the slab needs to be perfectly minimal.

Whichever you pick:

- Leave room for **airflow** around any heat-generating components.
- Provide a **service access path** for the ESP32 USB port, even if you primarily flash via OTA. You may need wired serial for debugging.
- Make sure the **power switch** (on the frame, not the slab) is reachable without crouching all the way under the table.

4.6 Installation: removable top

The gaming table sits on top of an existing bumper pool table, and the top slab needs to be liftable. The build splits into two parts that connect via a single **DC quick-disconnect**. Mains AC stays put on the frame; only +5V DC crosses the joint.

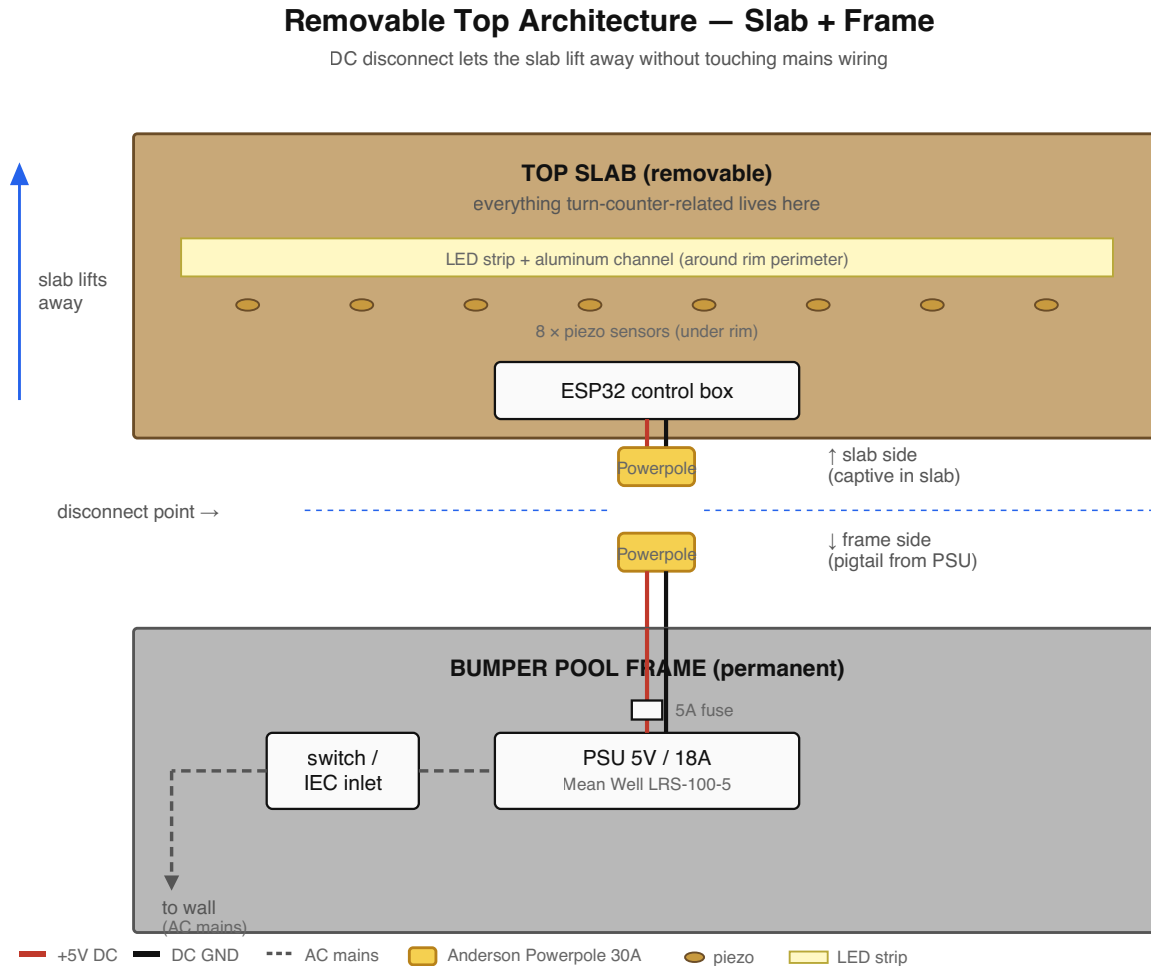


Figure 4.6 — slab/frame architecture. Everything turn-counter-related lives on the removable top slab; the PSU and AC mains live permanently on the bumper pool frame. A single Anderson Powerpole DC connector is the only thing you disconnect when lifting the top.

Why DC and not AC at the disconnect:

- **Safety.** AC mains at the disconnect point means a panel-mount connector that's potentially live every time you reach for it. Disconnecting mains at a casual touchpoint is a nope.
- **Practicality.** AC mains-rated panel connectors are bulky, expensive, and require strict polarity and earth bonding. Comparable DC connectors are cheap, robust, and idiot-proof.

What lives where:

Slab (top, removable): LED strip + channel, all 8 piezos, ESP32 control box, level shifter, all signal wiring, and a DC pigtail terminating in a Powerpole connector.

Frame (bottom, permanent): PSU, switch or IEC inlet, AC cord to wall, inline 5A fuse (DC side), DC pigtail terminating in the mating Powerpole connector.

The disconnect itself: Anderson Powerpole 30A connectors. They're polarized (can't connect backwards), genderless (the same housing mates with itself), rated comfortably above the 10A budget, and click together solidly. Red + black housing pairs are about \$5; the contacts are soldered and heat-shrunk rather than crimped (decision locked in — `shopping_list.md` has the per-contact procedure), so there's no crimp tool to buy. XT60 connectors (RC vehicle world) are \$2/pair and electrically equivalent, but the exposed bullet pins on the disconnected end aren't great if anyone ever pokes at them. Powerpole contacts are recessed.

Cable: 14 AWG silicone-insulated wire is overkill for 10A but stays flexible at the connector and is pleasant to work with. 16 AWG is the practical minimum. Run twisted pairs (red 5V + black GND) for tidy routing. Length: PSU location to slab + 18" of slack so you can lift the slab a foot before having to disconnect.

Slab cable entry: Buy the rubber grommet first, *then* drill the hole to match — most hardware-store grommets are spec'd by their *panel-hole size* (commonly $\frac{1}{2}$ " or $\frac{5}{8}$ "), and the grommet's inside diameter should snug-fit your DC cable (the 14 AWG silicone twisted pair is $\sim \frac{5}{16}$ " / 8 mm OD). Use a soft rubber grommet from the hardware store, not a sheet-metal-panel bushing like the Heyco 2092 — those snap-fit into 3 mm panels and won't retain in 1" wood. After installing the grommet, secure the cable inside the slab with a P-clip or adhesive cable mount within 4 inches of the entry — any pull on the cord should hit the clip, not the connector terminations.

PSU mounting: The Mean Well's mounting flanges screw to a small wood block ($\sim 4 \times 6 \times 1$ "), which then screws to the bumper pool frame underside in a ventilated location. Don't seal it inside an enclosed box — these run warm and need airflow.

Bench testing: When the slab is off the table, the lights have no power. Keep a spare Powerpole pigtail wired to a 5V/3A bench supply — plug into the slab's connector for moderate-brightness operation away from the table. Not enough current to drive the strip hard, but plenty for testing.

5. Phase-by-Phase Build Steps

Phase 1: Plan & Gather

- ☐ Phase 0 complete and Tap Light works (see §0)
- ☐ Confirm BOM and order remaining parts
- ☐ Print this doc and the wiring diagram (`turn_counter_wiring.svg`), keep them on the bench
- ☐ Measure the actual slab: confirm it's ~1" thick, all 8 sides equal length, lies flat on the bumper pool frame
- ☐ Identify a ventilated mounting location on the underside of the bumper pool frame for the PSU (and note where the AC outlet is relative to it)
- ☐ Plan the cable run from PSU → Powerpole → up to the slab. Mark roughly where the slab cable will enter
- ☐ Confirm you have wall outlet access within reach of the planned PSU location, ideally on a switchable surge protector

Phase 2: Bench Prototype

- ☐ Wire ESP32-S3 + 30-LED test segment + 1 piezo on breadboard per the wiring diagram (`doc-src/breadboard_layout.svg` shows a hole-by-hole placement). Connect the piezo to GPIO 1 (`PIEZO_PINS[0]`)
- ☐ Flash the main firmware with these temporary edits so the bench rig fits the small strip: `NUM_LEDS = 24` , `LEDS_PER_SIDE = 3` . Leave `NUM_SIDES = 8` and the default 4-player layout — each player owns 2 sides × 3 LEDs = 6 LEDs on the test strip. Tapping the wired piezo (side 0) will advance the lit zone from player 0 to player 1 once; after that, side 0 is no longer the active player's side and further taps will print *"Tap on side 0 ignored - not active player's side"* on Serial. That's the active-side filter doing its job — proof the firmware is wired up correctly. To watch the lit zone rotate through *all* 4 players on the bench, replace the `if (playerForSide(side) == currentPlayer) { advanceTurn(); renderTurn(); } else { Serial.printf(...); }` block in `commitTap()` with just `advanceTurn(); renderTurn();` and re-flash. Every tap will now advance the turn unconditionally. Restore the original block before moving to Phase 3
- ☐ Open Serial Monitor at 115200 baud
- ☐ Add a temporary `int v = analogRead(PIEZO_PINS[0]); if (v > 100) Serial.println(v);` inside `loop()` and tap the piezo — the threshold of 100 keeps the serial output quiet at idle and only prints when something interesting happens (note the single-read pattern: a piezo's voltage decays fast, so reading twice would give two different numbers)
- ☐ Note the reading at rest (probably 0–50) and the peak when tapped (likely 500–3000+)
- ☐ Set `TAP_DELTA` to ~30% of the peak reading's jump above the printed baseline — adjust by feel (the boot log prints each side's seeded baseline; a tap fires at baseline + `TAP_DELTA`)
- ☐ Confirm the LEDs change color/zone on tap before proceeding
- ☐ Test power-cycle persistence: tap a few times, unplug, replug, verify state restored

- ☐ If using OTA: configure Wi-Fi credentials in firmware, confirm device appears on the network and can be reached via `ping turn-counter.local`

Don't move forward until this works reliably.

Phase 3: LED Rim

- ☐ Measure all 8 sides — confirm they're actually 20", not 19.875" or something
- ☐ Cut aluminum channel: 8 pieces with 22.5° miters at each end
- ☐ Pre-fit the channel dry around the slab's outer edge, mark mounting screw locations
- ☐ Cut LED strip into 8 segments, ~30 LEDs each, between pad gaps
- ☐ Lay strips into channels, peel adhesive backing
- ☐ Solder jumper wires at each corner (3-conductor: 5V, GND, Data); heat-shrink each joint
- ☐ Test continuity: data line from start to end of the assembled strip with multimeter
- ☐ Mount channel pieces to the **outer edge of the slab** with screws (or recessed in a routed rabbet — see §4.1)
- ☐ **Count the actual installed LEDs** (most likely 232–240 depending on corner cuts) and update `NUM_LEDS` in firmware to match. Also recompute `LEDS_PER_SIDE = NUM_LEDS / 8`, **rounding down** (so e.g. 235 LEDs → 29 per side, with the last 3 LEDs unaddressed). Rounding up would make `renderTurn()` write past the end of the `leds[]` array
- ☐ Solder pigtails for power injection at three points along the strip: at the start (DIN end), at the corner roughly halfway around (the corner between the 4th and 5th side as the strip runs), and at the end of the strip. Each pigtail joins +5V and GND from the slab's main DC rails to the strip's 5V and GND pads

Phase 4: Piezo Mounting

- ☐ Solder leads to all 8 piezos (red to +, black to –). Quick — under 2 seconds per pad
- ☐ Test each piezo before mounting: connect to multimeter on AC voltage, tap it, expect a brief swing
- ☐ Splice each piezo's leads onto a pre-crimped JST-XH pigtail from the connector kit (solder + heat-shrink — the kit's wires come pre-crimped, so no JST crimper is needed)
- ☐ On the slab underside, mark 8 piezo locations: one per wedge, ~3–4" inward from each outer edge, avoiding any spot where the slab rests on the pool table frame
- ☐ Glue each piezo at its mark, brass face against wood
- ☐ Route piezo cables along the slab underside toward where the control box will live; secure with adhesive cable mounts every 6"

Phase 5: Control Box

- ☐ Build the protoboard: ESP32 socket (use female headers, don't solder it directly), level shifter, 8× pulldown resistors, 8× Zeners, 470 Ω data resistor, 1000 μF cap
- ☐ Add 8× JST-XH 2-pin headers for piezo inputs
- ☐ Add 1× JST-XH 3-pin header for strip data + ground reference
- ☐ Add a 5V/GND input pigtail (its far end lands in a WAGO 221 lever node on the slab DC rail)

- ☐ Test before installing: power up with the bench-test pigtail (see §4.6) connected to a 5V supply — either a small bench supply, or a USB-C wall adapter with a USB-C-to-Powerpole adapter cable, or even just the ESP32-S3's USB power if you only need to verify the firmware logic without driving the LED strip — then run firmware and tap piezo leads with a screwdriver to fake hits
- ☐ Mount protoboard inside the chosen enclosure (see §4.5 — typically a project box under the slab); cut cable entries for power and signal

Phase 6: Final Assembly

The build splits into frame-side and slab-side work — see §4.6 for the architecture.

Phase 6a — Frame side (permanent installation)

- ☐ Mount Mean Well PSU to the wood block with #8 screws through its mounting flanges
- ☐ Screw the wood block to the underside of the bumper pool frame, in a ventilated location
- ☐ Wire AC mains using the discrete-component approach (BOM default): wall plug → IEC C14 inlet → SPST rocker switch (interrupts Line only) → PSU's L (live) terminal. No fuse on the AC line — the inline blade fuse is DC-side only (see §2's wiring note; the PSU has its own internal input fuse). Neutral runs IEC inlet → PSU's N terminal **direct, with no switch** (the SPST switch has only one pole — it breaks Line only, same as every wall switch in a US household). Earth runs IEC inlet's Earth pin → PSU's chassis Earth lug, no switch on Earth. The IEC inlet and switch terminals are 0.250" faston tabs — either crimp female spade connectors onto the wire ends or solder directly with heat-shrink (3 on Line + 1 Neutral + 1 Earth = 5 faston connections total). Verify continuity with a multimeter (switch ON: Line continuous from inlet to PSU; switch OFF: Line open; Neutral and Earth always continuous) **before** plugging into the wall. Remember the safety habit from §2: the SPST switch breaks Line only, so **unplug from the wall before opening the enclosure** — only the wall plug guarantees both Line and Neutral are dead. If you'd rather interrupt both poles, swap the SPST for a DPST switch (wire Neutral through its second pole) or use a Schurter DG12 integrated inlet — see §2 wiring note
- ☐ Wire PSU's +V output through the inline 5 A fuse to the +V contact of a Powerpole housing; PSU's -V direct to the -V contact
- ☐ Solder Powerpole contacts onto silicone wire ends (procedure in `shopping_list.md`); insert into housings (red = +V, black = GND); slide the two housings together so they're locked
- ☐ Leave the cable long enough to reach the slab's connector with ~18" of slack
- ☐ First power test (frame only, no slab connected): switch on, multimeter the Powerpole, expect +5V between red and black
- ☐ Switch off before continuing

Phase 6b — Slab side (mobile assembly)

- ☐ Drill a 1/2" hole near the slab edge for the cable; install the rubber grommet
- ☐ Run the slab-side DC cable through the grommet into the slab
- ☐ Solder Powerpole contacts onto the slab-side cable ends; insert into mating housings (the genderless design means it's the same housing as the frame side)

- ☐ Inside the slab: branch from the slab-side DC cable to the control box's 5V/GND terminals AND to the 3 strip injection points (start, middle, end of strip). Branching method: land all incoming wires in the BOM's WAGO 221 lever connectors (cleanest), or twist-and-solder them with heat-shrink (works fine but harder to service)
- ☐ **Verify polarity at every junction with a multimeter before connecting.** Reversed 5V instantly kills the strip
- ☐ Secure the cable with a P-clip or adhesive cable mount within 4" of the grommet — strain relief should hit the clip, not the connector
- ☐ Connect all 8 piezo JST connectors to the control box
- ☐ Connect the strip data + ground JST
- ☐ Mount the control box; dress all wiring with cable clips; nothing dangling

Phase 6c — Mate and test

- ☐ **Slab bench test (before connecting to the frame):** with the slab on a workbench (sawhorses or table), connect the bench-test Powerpole pigtail to the slab's connector and to a 5V supply. Confirm the strip lights up and the firmware advances on tap. If anything's wrong, fix it now while access is easy
- ☐ Disconnect the bench supply
- ☐ Mate the slab Powerpole to the frame Powerpole
- ☐ First full power-up: switch on, watch for smoke (good sign: no smoke)
- ☐ Tap through every player position in turn (with the default 4-player layout, that's 4 advances; each player owns 2 contiguous sides). Look for dead LEDs, wrong colors, or visible gaps at corner joints — the lit zone should sweep cleanly across all 8 sides as you cycle through
- ☐ Test the disconnect: switch off → unmate Powerpole → lift the slab a foot → set it back → reconnect Powerpole → switch on → confirm everything works

Phase 7: Tune & Test

- ☐ Re-flash with `Serial.println` instrumentation enabled (over OTA if Wi-Fi is set up, or USB). For calibration, temporarily add a print of the raw `analogRead` values for *every* side that exceeds a low threshold (say, 100) — this captures both direct hits and cross-talk so you can see the cross-talk geometry
- ☐ Open the Serial Monitor (or for OTA, a network log viewer) and keep it visible while you tap
- ☐ **For the per-side reading test:** just tap each side firmly while watching the Serial Monitor. The instrumentation prints raw readings independent of the active-side filter, so you don't need to disable it — peaks for all 8 sides should be roughly similar
- ☐ If any side reads much lower, check the glue contact and lead solder joints
- ☐ Set `TAP_DELTA` to the value that ignores incidental table bumps but catches deliberate taps (typically ~30% of a deliberate tap's jump above baseline)
- ☐ **Cross-talk test:** tap side 1 hard, look at the serial output. The strongest reading should be side 1; sides 2 and 8 (adjacent) will show smaller readings; side 5 (opposite) should be near-zero. Confirm the firmware identifies the hit as side 1 — the only positive signal is the lit-zone advance (or the absence of an "ignored" log line for that side). If the firmware ever picks a non-adjacent side as the strongest, raise `TAP_DELTA` or add a relative-strength check (see §4.4)

- ☐ **On/off gesture test:** with two hands, slap two opposite sides simultaneously. Lights should toggle. If no toggle, check that both piezos spike above their baselines; raise `OPPOSITE_PAIR_WINDOW_MS` if your slap timing isn't quite synchronized
- ☐ **Setup gesture test:** rapidly tap an **unlit** side 4 times within 2 seconds. Strip should start blinking. Tap once more to cycle player count. Wait 3 seconds; strip resumes normal play at player 1
- ☐ Set `BRIGHTNESS` to your preferred level (start at 128, adjust)
- ☐ Play an actual game. Take notes on anything weird

6. Controls & Setup Mode

Once installed, the user-facing controls are:

Action	Result
Tap your own (lit) side during play	Advance turn to next player
Tap a side that <i>isn't</i> lit	Nothing — only the active player can pass turn
Rapid-tap an unlit side 4 times within 2 seconds	Enter setup mode (sides flash)
Tap two opposite sides simultaneously (a two-handed slap, on sides directly across from each other)	Toggle on/off
In setup: tap any side	Increment player count by one each tap, wrapping 8 → 2 (starts from whatever count was previously saved)
In setup: stop tapping for 3 s	Save player count, exit setup, reset to player 1
Power cycle	Resume on the same player and same on/off state (state persists)

Number of *blinking* sides = current player count selection.

The active-side rule: turn advance only triggers when the tapped side belongs to the currently lit zone. If player 3 is active, only taps within player 3's wedge advance the turn. This means players can't accidentally (or deliberately) advance someone else's turn by tapping their own section while waiting. It also reinforces "look at the lights to know whose turn it is" as the natural way to play. Non-active-side taps are silently ignored — no flash, no sound, the lit zone stays put.

This rule applies only to turn advance. The on/off gesture works from any sides (any opposite pair). Setup-mode entry, by contrast, counts **only taps on unlit sides** — so fast turn-passing (which always lands on the lit side) can never trip it. See below.

Why the entry gesture isn't a "hold": piezos detect vibration, not pressure. They produce a brief voltage spike when struck and then return to baseline — there's no signal to read while you keep your hand there. So setup entry uses a rapid tap burst instead. To keep a brisk round of normal play from looking like that burst, the firmware counts **only taps on unlit (non-active) sides** toward the gesture — turn-passes always land on the lit side, so they never accumulate. The natural way in is therefore to rapidly tap any dark side 4 times. If you instead start on your own lit section, the first tap just passes the turn (your side goes dark), and four more dark-side taps then trigger setup — i.e. it takes one extra tap from your own seat. Exiting setup resets to player 1 anyway, so that initial pass doesn't matter.

The on/off gesture in detail: any two-handed slap on diametrically opposite sides toggles the LEDs. The firmware buffers each tap for 150 ms before committing it as a turn advance — long enough to detect a near-simultaneous opposite tap, short enough that the resulting delay on normal turn passes is barely perceptible. Off state is fully dark; the device still polls the piezos and watches for the on-gesture, but ignores everything else. The 150 ms buffer also handles the asymmetric-strike case where one hand lands a few milliseconds before the other.

Why opposite sides specifically: cross-talk through the wooden slab falls off with distance. Adjacent piezos (sides 1 + 2) might both register from a single hard hit on side 1. Opposite piezos (sides 1 + 5) physically can't — they're as far apart as possible in the slab, so two strong simultaneous readings on opposites can only come from a deliberate two-handed slap.

7. Firmware

The main firmware is in `turn_counter.ino`. The Phase 0 starter firmware is in `tap_light.ino`. Key tunables at the top of the main file:

Constant	Default	Tune to...
<code>NUM_LEDS</code>	240	Actual LED count after corner trim
<code>LEDS_PER_SIDE</code>	30	$\text{NUM_LEDS} / 8$
<code>BRIGHTNESS</code>	128	Lower if too bright, max 255
<code>TAP_DELTA</code>	1000	Calibrate during Phase 7. Adaptive: a tap fires at a side's auto-tracked baseline + this delta
<code>DEBOUNCE_MS</code>	250	Raise if double-triggers, lower if it feels sluggish
<code>SETUP_TAP_COUNT</code>	4	Taps required to enter setup mode
<code>SETUP_TAP_WINDOW_MS</code>	2000	Window in which the taps must occur
<code>SETUP_EXIT_IDLE_MS</code>	3000	Idle time in setup mode before saving and exiting
<code>OPPOSITE_PAIR_WINDOW_MS</code>	150	Window for detecting the on/off two-handed gesture; raise to make it more forgiving, lower for snappier turn passes
<code>WIFI_SSID /</code> <code>WIFI_PASSWORD</code>	placeholder	Your home Wi-Fi (for OTA)
<code>OTA_HOSTNAME</code>	<code>turn-counter</code>	mDNS name; reach device at <code>turn-counter.local</code>
<code>OTA_PASSWORD</code>	<code>change-me</code>	Required to push firmware updates

`PLAYER_COLORS[]` array defines the color for each player — edit to taste.

Build environment: Arduino IDE with ESP32 board support, FastLED library installed (ArduinoOTA is bundled with the ESP32 core). Board: **"ESP32S3 Dev Module"**. **USB CDC On Boot: Enabled** (so Serial works over the native USB port). Upload speed: 921600.

7.1 OTA firmware updates

The device tries to join Wi-Fi at boot with a 5-second timeout. If it joins, OTA is available; if not, it just runs locally — the device works fine without a network.

To push an update from Arduino IDE:

1. With the device powered and on the same network as your computer, the IDE's "Port" menu will list `turn-counter` at `192.168.x.x` (or whatever the mDNS resolves to) under "Network Ports".
2. Select that port, hit Upload, enter the OTA password when prompted.
3. The strip goes dark, then a blue progress bar fills around the rim as the update transfers.
4. On success, all LEDs flash green briefly. On failure, all red for 2 seconds.

5. Device reboots into the new firmware.

Security note: anyone on your local network can attempt to flash the device. The OTA password protects against accidental or casual attacks but isn't strong security. Don't put this on a network with untrusted devices, and definitely don't expose it to the internet.

If OTA breaks (most common cause: pushing firmware that crashes immediately and loses Wi-Fi): you can always fall back to USB. Plug into the ESP32-S3, hit Upload, done. Keep a USB cable accessible.

8. Troubleshooting

Symptom	Likely cause	Fix
LEDs flicker or first LED is wrong color	Missing or weak data signal	Confirm 470 Ω resistor + level shifter + common ground
Far-end LEDs tint pink/orange	Voltage drop along strip	Add or check power injection at midpoint and end
Whole strip dark	PSU off, switch off, blown fuse, or reversed polarity	Check switch first, then fuse, then PSU output, then polarity
Tap doesn't register	TAP_DELTA too high, bad piezo solder, glue not contacting wood	Lower TAP_DELTA , reflow joint, re-glue
Tap on side 1 lights side 3	Piezo wire mapping wrong	Check PIEZO_PINS [] order vs physical wiring
Adjacent sides cross-trigger	Mechanical cross-talk through wood	Foam break, kerf cut, or relative-strength filter in firmware
Weird boot behavior	Strapping pin pulled wrong	Confirm GPIO 0/3/45/46 unused (ESP32-S3 strap pins)
ESP32-S3 won't flash via USB	Upload speed, USB CDC setting, or held button	Drop to 115200 baud, confirm Tools → USB CDC On Boot is Enabled, hold BOOT + tap RESET + release BOOT to enter download mode manually
OTA port doesn't appear in IDE	Wi-Fi didn't join, wrong network, mDNS not resolving	Check Serial Monitor at boot for IP; ping turn-counter.local ; fall back to USB
OTA fails midway	Network drop or insufficient flash	Retry; consider partition scheme with larger OTA region
Player count resets unexpectedly	NVS partition issue	Erase flash and re-flash: <code>esptool.py erase_flash</code>
Solder joint won't take	Tip too cold, no flux, dirty surface	Increase iron temp, add flux pen, clean the surface with isopropyl
Solder joint looks dull / blobby	Cold joint — moved while cooling, or insufficient heat	Reflow with flux, hold steady ~2 sec while cooling

9. Future Enhancements

If the basic build lands well, things worth considering later:

- **Turn timer:** each player has N seconds; the lit zone slowly drains around the side as time runs out. Tap before it empties or it auto-advances with a red flash.
- **Companion app:** ESP32-S3 already has Wi-Fi up for OTA. Run a small web server, expose a phone-friendly page for round counting, scoring, initiative tracking. Especially useful for RPGs. (BLE 5.0 is also on the table — the S3 has both radios, and ADC1 is unaffected by either.)
- **Audio feedback:** a small piezo *buzzer* (different from the sensor piezos) on a free GPIO for soft turn-pass clicks.
- **Per-game profiles:** store named configurations in NVS. "Magic 4-player", "RPG initiative for 6", etc.
- **Initiative mode:** instead of round-robin, player order is a stored sequence. Useful for D&D-style turn order that isn't seat-based.
- **Color customization:** the companion app could let players pick their own colors before the game starts.
- **Round counter:** subtle flash or pulse when a full round completes. Easy to add given current architecture.

10. Quick Reference Card

Keep this part handy at the table:

To pass turn: tap the lit section in front of you. Only the active player can advance. **To turn on/off:** slap any two opposite sides of the table at the same time (a two-handed gesture: left hand on one side, right hand on the side directly across). **To change player count:** rapid-tap an unlit (dark) side 4 times within 2 seconds. The lights will start blinking. Then tap to cycle through counts. Stop tapping for 3 seconds to save. **To reset to player 1:** enter setup mode and exit (count stays the same, turn resets). **State persists across power cycles** — including on/off, current player, and player count.

End of document.